TEAM DEMO — PHASE 1

Questionnaire Platform

A backend-driven system for collecting structured personal and family-history data for genetic-risk assessment. This document covers the confirmed design end-to-end, then draws a hard line at exactly what's being implemented next.

Spring Boot 4.1 · Java 21 | PostgreSQL · Liquibase | JSONLogic rule engine | Prepared by Safiya Khan

01 What are we building?
02 Architecture
03 Questionnaire hierarchy
04 Data model
05 Rules & execution
06 APIs being implemented now
07 Status & next steps
+ Backup material

01 What are we building?

A workflow-driven questionnaire platform that collects structured personal and family-health information. The current build is deliberately scoped to **collection and persistence only** — everything downstream is future work, shaped for but not part of this phase.

PHASE 1 — CURRENT

1. Questionnaire
2. Collect data
3. Export JSON

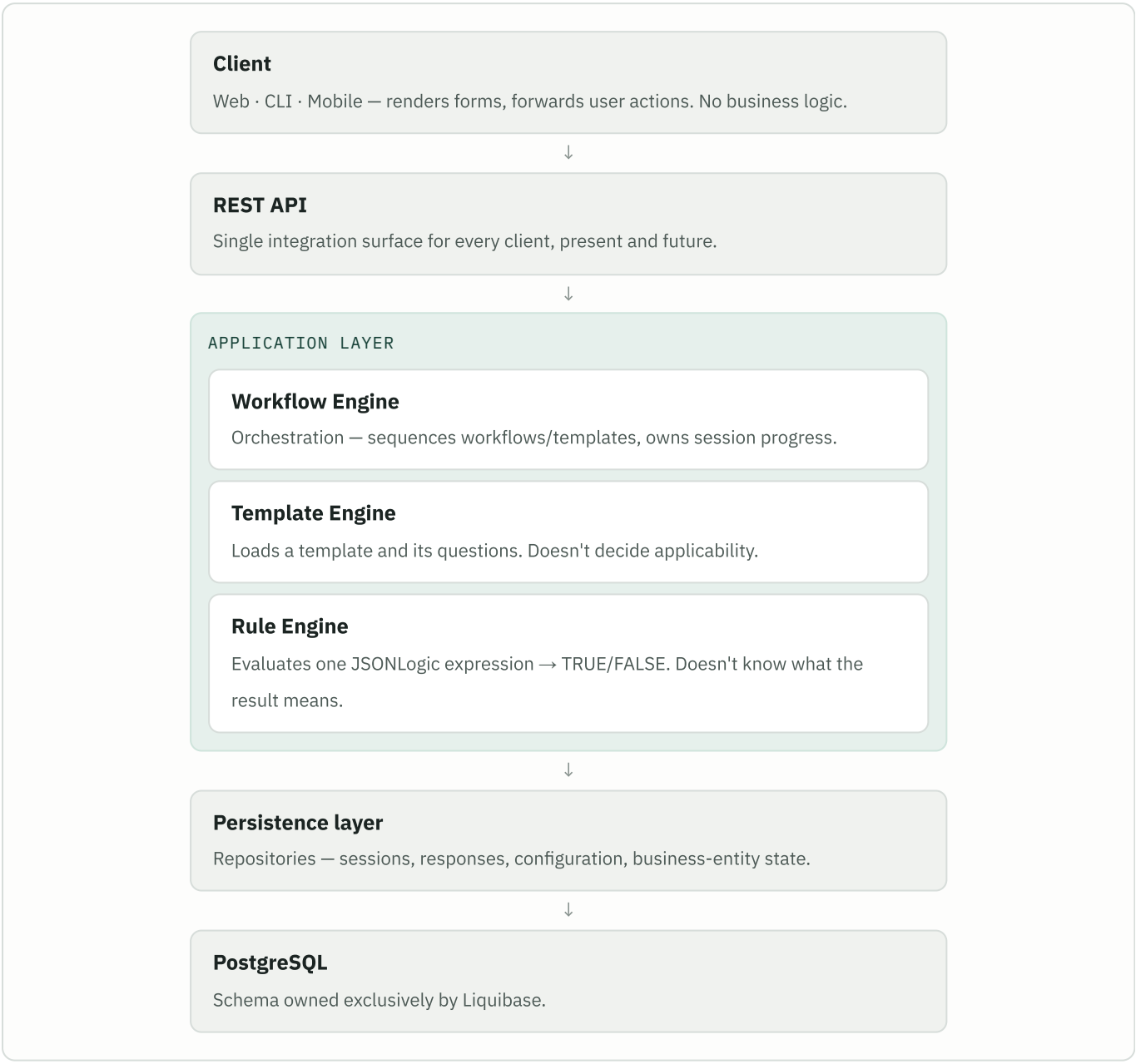
FUTURE PHASES

1. Pedigree system
2. Risk assessment
3. Recommendations
4. PDF report

- **Backend-driven.** All business logic, navigation, and validation live on the server. The client only renders and forwards input — "the client shouldn't think, the server should think."
- **Configuration, not hardcoding.** Questions, options, validation and rules are data, so questionnaire behaviour can change without a code deploy.
- **Save & resume by default.** Every answer is persisted immediately, so an interrupted session picks up exactly where it stopped.

02 Architecture

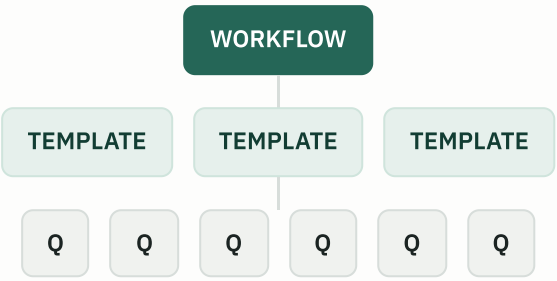
A thin client, a single REST surface, and three single-responsibility engines sitting in front of persistence.



> **The one sentence to remember:** the client renders, the server decides.

03 Questionnaire hierarchy

Every question the platform ever asks sits under exactly one template, which sits under exactly one workflow.



EXAMPLE — BUILT VIA THE APIS IN §06, NOT YET SEEDED



COMPONENT	RESPONSIBILITY
Workflow	Major questionnaire section (e.g. Personal Details).
Template	A reusable, ordered set of questions within a workflow.
Question	A single data-collection item — type, options, validation.
Rule	Determines applicability, at template or question level.
Response	The submitted answer, stored as an append-only record.

04 Data model

Nine tables in three groups: what the questionnaire *is* (configuration), what a person is currently *doing* (execution), and what's actually *true about them* (business state).

CONFIGURATION	EXECUTION	BUSINESS STATE
Workflow → Template → Question	**Session → Response**	**Person / Family Member → Disease / Mutation**

TABLE	PURPOSE
`WORKFLOW`	A questionnaire section, ordered by `display_order`.
`TEMPLATE`	A reusable question set inside a workflow. Lowest `display_order` is the default/entry template. Carries `template_rule` (JSONB).
`QUESTION`	One question. Carries `options` / `validation` inline, plus one rule: `visibility_rule`.
`PERSON`	The proband. Carries `additional_facts` (JSONB) for anything without a fixed column.
`FAMILY_MEMBER`	A relative, linked via `proband_id`. Same shape as `PERSON`, plus `additional_facts`.
`QUESTIONNAIRE_SESSION`	One assessment run. `status` is `IN_PROGRESS` / `COMPLETED`; `previous_template_id` / `previous_workflow_id` track resume position.
`QUESTION_RESPONSE`	Append-only answer log. Carries `answer_key` and `template_id` directly; `relationship_instance_id` disambiguates repeated family members.
`DISEASE`	A diagnosis, owned by exactly one of `PERSON` / `FAMILY_MEMBER` — enforced by a DB check constraint.
`MUTATION`	A genetic finding, same single-owner constraint as `DISEASE`.

SIX ENUMS BACK EVERY FIXED-VOCABULARY COLUMN

ENUM	VALUES
`QuestionType`	TEXT, NUMBER, DATE, RADIO, CHECKBOX, DROPDOWN
`SessionStatus`	IN_PROGRESS, COMPLETED
`AnswerType`	BOOLEAN, NUMBER, STRING, ARRAY, OBJECT
`GenderAtBirthType`	MALE, FEMALE, INTERSEX, UNKNOWN
`MaternalPaternalSideType`	MATERNAL, PATERNAL, UNKNOWN
`RelationshipType`	18 values — MOTHER, FATHER, SIBLING/HALF-SIBLING variants, children (incl. adopted), SPOUSE, GRANDPARENTS, UNCLE/AUNT, COUSIN, NIECE/NEPHEW

05 How rules & execution work

One generic rule engine, interpreted two different ways depending on who's asking. A question carries exactly one rule type — show/hide.

	TEMPLATE RULE	QUESTION RULE
Decides	Whether a template loads or is skipped	Whether a question is shown
Interpreted by	Workflow Engine	Question/form handling logic
TRUE means	Load the template	Show the question
FALSE means	Skip to the next candidate template	Hide the question

EXAMPLE — A QUESTION'S VISIBILITY_RULE

```
{
  "==": [
    { "var": "gender_at_birth" },
    "FEMALE"
  ]
}
```

Answer flow

We persist the answer and update the business entity. Nothing more happens within the current template.

1. Submit answer

↓

2. Validate

↓

3. Persist QUESTION_RESPONSE

stored value, not the display label

↓

4. Update PERSON / FAMILY_MEMBER

> No re-evaluation happens within the current template — ever. A rule may only reference an **earlier** template's answers, never its own (see Template dependency, below), so nothing currently on screen can depend on an answer just given on that same screen. Re-evaluation only happens when the *next* template loads, against everything collected so far.

Template dependency

A rule may reference answer_keys from any earlier template — never its own — enforced as an authoring convention, not runtime-checked code. This is also why a question needs only one rule type: nothing in the same template can ever change as a result of another answer on that same screen.



✓ Template 5 may depend on Template 1, 2, 3, or 4 — any earlier template, not just the one right before it

✗ Template 5 may not depend on Template 5's own questions

06 What APIs are being implemented now

The configuration layer — Workflow → Template → Question. Assessment-execution APIs (session start, answer submission, navigation) come in the following phase and will read what these APIs create.

> **This is the exact boundary of the next implementation phase.** Everything above this line is design; everything below is the following phase.

METHOD	PATH	PURPOSE
POST	/admin/workflows	Create a workflow
GET	/admin/workflows	List all, by displayOrder
GET	/admin/workflow/{id}	Get one
PUT	/admin/workflow/{id}	Update
DELETE	/admin/workflow/{id}	Delete — blocked if templates exist
POST	/admin/workflow/{workflowId}/templates	Create a template
GET	/admin/workflow/{workflowId}/templates	List templates in a workflow
GET	/admin/template/{templateId}	Get one
PUT	/admin/template/{templateId}	Update, incl. templateRule
DELETE	/admin/template/{templateId}	Delete — blocked if questions exist
POST	/admin/template/{templateId}/questions	Create a question
GET	/admin/template/{templateId}/questions	List questions in a template
GET	/admin/question/{questionId}	Get one, full JSONB payload
PUT	/admin/question/{questionId}	Update
DELETE	/admin/question/{questionId}	Delete

All paths prefixed `/api/v1`. Plural for collection operations, singular once a specific ID is addressed.

Sample request bodies

`POST /ADMIN/WORKFLOWS`

```
{
  "code": "PERSONAL_DETAILS",
  "name": "Personal Details",
  "description": "Collect personal details",
  "displayOrder": 1
}
```

`POST /ADMIN/WORKFLOW/{ID}/TEMPLATES`

```
{
  "code": "WOMEN_DETAILS",
  "name": "Women Details",
  "displayOrder": 2,
  "templateRule": {
    "==": [{"var":"gender_at_birth"},"FEMALE"]
  }
}
```

`POST /ADMIN/TEMPLATE/{ID}/QUESTIONS`

```
{
  "answerKey": "gender_at_birth",
  "text": "What is your gender at birth?",
  "type": "RADIO",
  "displayOrder": 1,
  "options": [
    { "label": "Male", "value": "MALE" },
    { "label": "Female", "value": "FEMALE" }
  ],
  "validation": { "required": true },
  "visibilityRule": null
}
```

Layering decisions for this phase

Errors	One structured body on every failure — `{status, error, message, path, timestamp}` — via a single global exception handler.
Layering	Controller → Service → Repository. Controllers stay HTTP-only; the Service owns transactions and business rules.
Request/response bodies	DTOs, not raw JPA entities — keeps the API contract decoupled from the schema.
Deleting a Workflow	Blocked (409) while it still has templates — protects the append-only response history underneath.

07 Status & next steps

What's actually built and verified today, versus what's next.

Design docs studied and reconciled into one living SRS	DONE
Architecture finalized — REST → Workflow/Template/Rule Engine → Persistence	DONE
Data model finalized — 9 tables, 6 enums	DONE
JPA entities + Liquibase migrations written and verified against a live PostgreSQL schema	DONE
Single-rule JSONLogic model defined (template_rule / visibility_rule) — no re-evaluation within a template	DONE
Session resume & append-only response model defined	DONE
Template dependency rule defined (any earlier template, never its own)	DONE
Workflow / Template / Question admin APIs — implemented and live-tested	DONE
Assessment core loop — start/resume, current-template, answer, next-template, status, responses — implemented and live-tested	DONE
Family-member management, previous/forward browsing + edit cascade, signed-cookie session id, returning-patient matching	NEXT

+ Backup material

Resolved, documented, and ready if asked — kept out of the main walkthrough on purpose.

Session identification & returning patients

The active session is carried via a signed cookie (never a URL parameter); `personId` is never exposed client-side. A returning proband is matched via a lightweight identifying field (email/phone) with explicit confirmation before linking — never automatic. Not yet implemented — the current build identifies a session by a plain field/path parameter.

Previous / forward browsing and editing

Browsing a past template is read-only and never touches real progress. Editing requires an explicit "go back to here" action that moves the real session position, after which the normal answer-submission path applies. "Nullifying" an invalidated answer clears the business-entity field only — the historical response record is untouched. Not yet implemented.

Duplicate & existing-family-member handling

Singular relationships (mother, father) are capped at one record; grandparents are capped per maternal/paternal side. Plural relationships (siblings, cousins) are reconciled against a declared count rather than de-duplicated by type. A family member who later becomes their own proband is linked via a deterministic reference token — never fuzzy name/DOB matching, given the risk of mis-linking two people's health records. Not yet implemented — answers currently only ever apply to the proband.

Answer validation structure

The `validation` JSONB uses a fixed, closed set of named keys — `required`, `minLength` / `maxLength`, `min` / `max`, `pattern` — rather than an embedded rule expression, validated server-side only.

Sources: QP-Understanding, QP-SRS (v1.4), QP-Post-HLD Implementation Context, QP-Execution Flow, QP-APIs List, Final Conclusions, QP-DataModel. Status reflects verified state as of the disable_rule removal / single show-hide rule decision (2026-09-21).